2.2. Utilización de los distintos tipos de variables y operadores disponibles en el lenguaje (Criterio 2.2)

- **Resultado de Aprendizaje 2 (RA2):** *Escribe sentencias simples, aplicando la sintaxis del lenguaje y verificando su ejecución sobre navegadores Web.*

- **Criterio Curricular Oficial: CE 2.b** (*Se han utilizado los distintos tipos de variables y operadores disponibles en el lenguaje*).

- **Ponderación del Criterio:** (**10% del RA2** | **0,5% sobre la calificación final del módulo**).

En JavaScript, el manejo de datos se apoya en un sistema de tipado débil y dinámico: los tipos están asociados a los valores y no a las variables que los contienen. Un identificador puede albergar un número entero en un instante de la ejecución y contener una cadena de texto o un objeto complejo líneas más tarde.

Aclaración Previa: ¿5 o 7 tipos primitivos?

En los manuales tradicionales de JavaScript basados en el estándar clásico ECMAScript 5 se suele enseñar que existen **5 tipos primitivos básicos**:

1. number
2. string
3. boolean
4. undefined
5. null

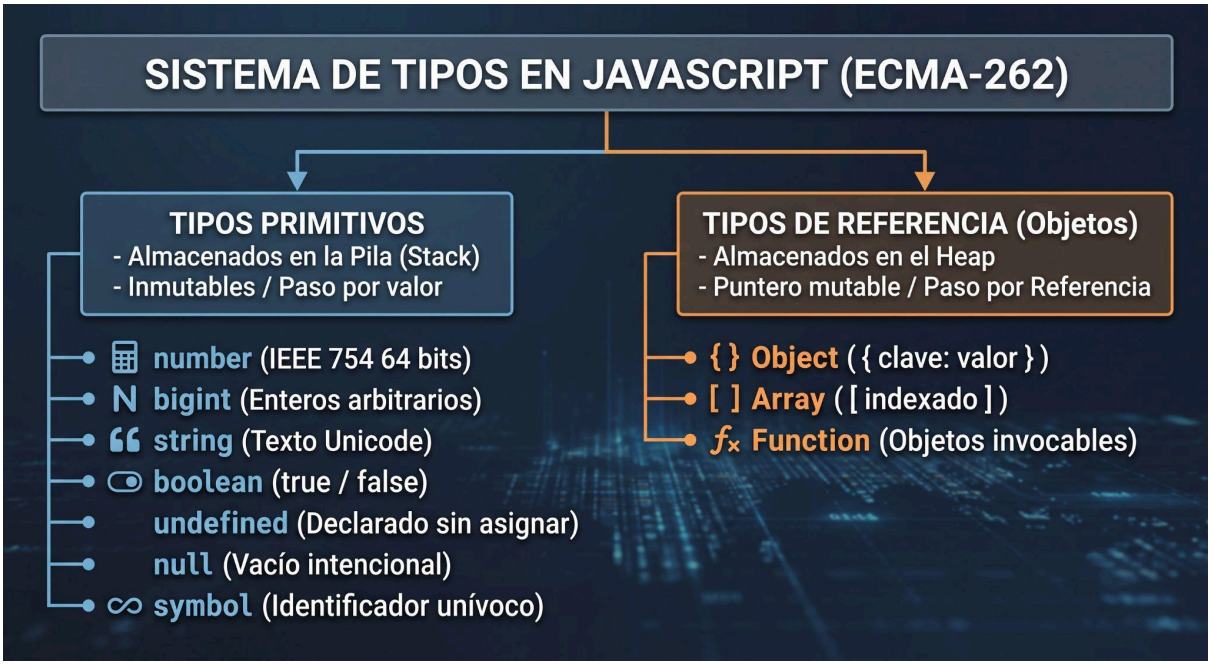
Sin embargo, el estándar moderno (ECMAScript 6 y versiones posteriores) incorporó **2 nuevos tipos primitivos** para resolver problemas técnicos concretos:

6. symbol (ES6 - 2015): Identificadores únicos que no chocan entre sí.

7. bigint (ES2020): Números enteros de longitud arbitraria que superan los límites de cálculo de number.

Por tanto:

- **En el estándar clásico (ES5):** Hay **5 tipos primitivos**.
- **En el estándar moderno (ES6+):** Hay **7 tipos primitivos** y **1 tipo de referencia** (Object, del cual derivan los arrays y las funciones).



Tipos Primitivos: Por Valor e Inmutables

Los tipos primitivos (`number`, `string`, etc.) se guardan **por valor directo** en una memoria rápida llamada *Stack*.

- **¿Qué significa "Por Valor"?** Que cuando copias una variable a otra, JavaScript hace una **fotocopia exacta**. Ambas variables se vuelven totalmente independientes.
- **¿Qué significa "Inmutable"?** Que el valor original **no se puede modificar**, solo se puede sustituir por uno nuevo.

Ejemplo de comportamiento por valor:

```
let precioOriginal = 100;
let precioCopia = precioOriginal; // Se hace una "fotocopia" del número
100

precioCopia = 150; // Cambiamos la copia

console.log(precioOriginal); // Imprime: 100 (Sigue intacto)
console.log(precioCopia);    // Imprime: 150
```

Ejemplo de Inmutabilidad:

Si tienes el string `"hola"`, no puedes cambiar mágicamente su primera letra para que diga `"bola"`. Tienes que crear un string completamente nuevo.

```
let texto = "hola";
console.log(texto[0]); // Imprime: "h"
```

```
texto[0] = "b"; // JavaScript ignora esto silenciosamente
console.log(texto); // Imprime: "hola" (No cambió)
```

Activa el "Strict Mode" (modo estricto) para que JavaScript te avise con un error cuando intentes hacer esto

```
"use strict"; // Activamos el modo estricto let texto = "hola";
texto[0] = "b"; // ¡Aquí el programa se detiene!
console.log(texto);
```

2. Tipos de Referencia: Por Referencia y Mutables

Los objetos, arrays y funciones son complejos y pesados, por lo que se guardan en una memoria más grande llamada *Heap*. En la variable no se guarda el objeto real, sino una **dirección de memoria** (como un mapa o un enlace web).

- **¿Qué significa "Por Referencia"?** Que cuando copias una variable a otra, **no duplicas el objeto**. Lo que estás copiando es el enlace a la misma dirección de memoria. Ambos apuntan al mismo lugar.
- **¿Qué significa "Mutable"?** Que puedes alterar, añadir o borrar partes del interior del objeto o array sin cambiar su dirección de memoria.

Ejemplo de comportamiento por referencia y mutabilidad:

Imagina que creas un array y lo "copias":

```
const lista = ["HTML", "CSS"];
const listaCopia = lista; // ¡Ojo! No es una copia
listaCopia.push("JS"); // Modificamos (mutamos) el array a través de la copia

console.log(lista); // Imprime: ["HTML", "CSS", "JS"]
console.log(listaCopia);    // Imprime: ["HTML", "CSS", "JS"]
```



A. Declaración y Asignación de Variables: var, let y const

Una variable es un contenedor con nombre en la memoria del ordenador. JavaScript asocia el tipo de dato al **valor** que contiene y no al nombre de la variable (tipado dinámico).

1. Peculiaridades de la Declaración Clásica con var

var es la instrucción original de JavaScript (1995). Tiene particularidades que hoy se consideran problemáticas:

A. Declaración múltiple en una única sentencia

Se pueden declarar varias variables a la vez separándolas con comas:

```
// Se declaran tres variables en una misma línea
var x = 5, y = 6, z = x + y;
console.log(z); // Imprime: 11
```

B. Variable declarada pero no inicializada (undefined)

Si reservas la variable pero no le pones ningún valor, JavaScript le asigna automáticamente undefined:

```
var miCoche;
console.log(miCoche); // Imprime: undefined (existe, pero está vacía)
```

C. Redeclaración permitida sin avisar de error

Con var, si vuelves a escribir var con el mismo nombre y no le das un nuevo valor, la variable no se borra, conserva el valor que ya tenía:

```
var nombre = "Juan";
var nombre; // No da error y NO borra el contenido
```

```
console.log(nombre); // Imprime: "Juan"
```

D. Tipado dinámico en acción

Una misma variable puede cambiar de tipo de dato a lo largo del tiempo sin que el navegador proteste:

```
var dato;         // Tipo: undefined (sin valor aún)
dato = 5;         // Ahora su tipo es un número (number)
dato = "John";    // Ahora ha mutado a texto (string)
console.log(dato); // Imprime: "John"
```

2. Declaraciones Modernas: let y const (Recomendadas)

let (Para variables cuyo valor va a cambiar)

- **Ámbito de bloque:** Solo vive dentro de las llaves { ... } donde se crea ({}, un if, un for, una función).
- **Prohíbe la redeclaración:** Si intentas declarar dos veces la misma variable en el mismo bloque, el programa se detiene con un error sintáctico.
- **Permite reasignación:** Su valor puede variar.

```
let contador = 0;
contador = 1; // Totalmente válido: reasignamos su valor

// Ámbito de bloque con let:
{
  let mensajeSecreto = "Clave123";
  console.log(mensajeSecreto); // Funciona dentro de las llaves
}
// console.log(mensajeSecreto);
// ERROR: ReferenceError: mensajeSecreto is not defined (fuera de las llaves no existe)
```

const (Para valores fijos que no deben cambiar)

- **Obligatorio inicializar:** No puedes poner const pi;, debes darle su valor en la misma línea.
- **No permite reasignación:** Si intentas cambiar su valor, el navegador lanza un TypeError.
- **Convención:** Se suelen escribir en MAYÚSCULAS cuando son constantes fijas del programa.

```
const IVA = 0.21;
IVA = 0.10; // ERROR: TypeError: Assignment to constant variable
```

El matiz de const con Objetos y Arrays (Mutabilidad de contenido)

const impide cambiar la **referencia** (la "caja" en memoria), pero **sí permite alterar las propiedades de dentro**:

```
const usuario = { nombre: "Marcos", edad: 20 };
```

```
// VÁLIDO: Cambiamos una propiedad de dentro del objeto
usuario.nombre = "Laura";
console.log(usuario.nombre); // Imprime: "Laura"

// INVÁLIDO: No podemos sustituir la caja entera por otra
// usuario = { nombre: "Pedro" }; // ERROR: TypeError: Assignment to constant variable
```

B. Tipos de Datos Primitivos en Detalle (Explicados desde Cero)

1. Numéricos (number y bigint)

number (Para números enteros y decimales)

En JavaScript no existen los tipos int o float por separado: todos los números son del tipo number (formato de 64 bits en coma flotante según la norma IEEE 754).

```
let edad = 25;       // Entero
let precio = 19.99;  // Decimal (se usa punto, no coma)

// Notación científica:
let particulas = 2e-9; // Equivale a 2 * 10^-9 en física

// Bases numéricas con prefijos:
let binario = 0b101;     // El número 5 en binario (0b)
let octal = 0o12;        // El número 10 en octal (0o)
let hexadecimal = 0xFF;  // El número 255 en hexadecimal (0x)
```

Valores numéricos especiales:

- **Infinity / -Infinity:** Aparece al dividir entre cero en lugar de bloquear el programa:

  ```
  let division = 10 / 0;
  console.log(division); // Imprime: Infinity
  ```

- **NaN (*Not-a-Number*):** Aparece cuando intentas hacer una cuenta matemática con algo que no es un número:

  ```
  let errorCalculo = "hola" * 2;

  console.log(errorCalculo); // Imprime: NaN
  console.log(Number.isNaN(errorCalculo)); // Imprime: true (comprueba si es NaN)
  ```

El problema de la precisión en coma flotante (IEEE 754)

Al basarse en representación binaria de 64 bits de doble precisión, JavaScript no puede representar de forma exacta ciertas fracciones decimales periódicas en base 2:

```
console.log(0.1 + 0.2); // 0.30000000000000004
console.log(0.1 + 0.2 === 0.3); // false
```

- **Solución profesional:** Para comparar importes monetarios o cálculos críticos se recurre al redondeo explícito (Number.EPSILON o toFixed()):

```
Math.abs((0.1 + 0.2) - 0.3) < Number.EPSILON; // true
```

bigint (Para números enteros gigantes)

Los números normales pierden precisión si superan el valor $9.007.199.254.740.991$ (Number.MAX_SAFE_INTEGER). Si le añades una letra n al final a un número entero, se convierte en un bigint y no pierde ningún dígito:

```
let enteroGigante = 9007199254740991n + 5n;
console.log(enteroGigante); // Imprime: 9007199254740996n (cálculo exacto)
```

2. Cadenas de Texto (string)

Representa secuencias de texto. Se pueden crear de tres formas:

```
let saludo1 = 'Hola con comillas simples';
let saludo2 = "Hola con comillas dobles";

// Template literals (con acento grave `): Permiten meter variables dentro con ${}
let modulo = "DWEC";
let horas = 160;
let frase = `El módulo de ${modulo} tiene ${horas} horas lectivas.`;
console.log(frase); // Imprime: "El módulo de DWEC tiene 160 horas lectivas."
```

Secuencias de escape (Caracteres especiales que no se pueden teclear directamente):

- \n: Inserta un salto de línea.
- \t: Inserta una tabulación.
- \" o \': Permite meter comillas dentro de comillas del mismo tipo.
- \\: Dibuja una barra invertida.
- \u{20BF}: Inserta un símbolo Unicode mediante su código hexadecimal (ej. Bitcoin ₿).

```
console.log("Línea 1\nLínea 2 con tabulador:\t\u{20BF}");
```

3. Lógicos o Booleanos (boolean)

Solo admiten dos valores posibles: true (verdadero) o false (falso). Se usan para comprobar condiciones:

```
let tieneCarnet = true;
let esMayorDeEdad = false;
```

4. undefined frente a null (Diferencia visual y práctica)

- **undefined:** Significa que algo **no ha sido asignado todavía** (el sistema no sabe qué hay ahí).
- **null:** Es un valor que pones **tú voluntariamente** para indicar que esa variable está expresamente vacía.

```
let cajaSinAbrir;
```

```
console.log(cajaSinAbrir); // undefined (el motor ve que no tiene valor asignado)

let cajaVacia = null;
console.log(cajaVacia); // null (el programador indica: "esta caja está vacía a propósito")
```

5. symbol (Explicado paso a paso)

Un **symbol** es un identificador que el navegador garantiza que es **100% único en el universo de la memoria**. Aunque crees dos símbolos con la misma descripción, nunca serán iguales:

```
// Creamos dos identificadores:
let id1 = Symbol("matricula");
let id2 = Symbol("matricula");

// Comprobamos si son iguales:
console.log(id1 === id2); // Imprime: false (son completamente diferentes e independientes)

// ¿Para qué sirve? Para añadir identificadores ocultos a objetos sin peligro de sobreescribir
// propiedades que ya existieran en librerías de terceros:
let alumno = { nombre: "Elena" };
alumno[id1] = "EXP-2026-001";
console.log(alumno[id1]); // "EXP-2026-001"
```

C. El Operador de Inspección de Tipos: typeof

Se coloca delante de cualquier valor o variable para que nos diga qué tipo de dato es:

```
console.log(typeof 42);            // "number"
console.log(typeof "Hola");        // "string"
console.log(typeof true);          // "boolean"
console.log(typeof undefined);     // "undefined"
console.log(typeof 100n);          // "bigint"
console.log(typeof Symbol("id"));  // "symbol"
console.log(typeof { a: 1 });      // "object"
console.log(typeof [1, 2, 3]);     // "object" (los arrays son un tipo de objeto)
console.log(typeof function(){});  // "function" (las funciones son objetos ejecutables)

// Puntos conflictivos en exámenes y pruebas:
console.log(typeof NaN);           // "number" (aunque signifique Not-a-Number)
console.log(typeof null);          // "object" (Error histórico de 1995 de JavaScript que se conserva)
```

D. Operadores del Lenguaje (Con Ejemplos Claros)

1. Operadores Aritméticos

Sirven para realizar cálculos matemáticos:

Operador	Nombre	Ejemplo	Resultado
+	Suma / Concatenación	10 + 5	15 (si es texto: "10" + "5" da "105")
-	Resta	10 - 5	5
*	Multiplicación	10 * 5	50
/	División	10 / 4	2.5
%	Módulo (resto entero de la división)	10 % 3	1 (porque 10 entre 3 es 3 y sobra 1)
**	Exponenciación (potencia)	2 ** 3	8 ($2 \times 2 \times 2$)
++	Incremento unitario (+1)	let a = 5; a++;	a pasa a valer 6
--	Decremento unitario (-1)	let a = 5; a--;	a pasa a valer 4

Diferencia entre pre-incremento (++a) y post-incremento (a++):

```
let x = 5;
let y = x++; // Post-incremento: PRIMERO asigna el valor actual (5) a 'y', y LUEGO suma 1 a 'x'
console.log(y); // 5
console.log(x); // 6

let a = 5;
let b = ++a; // Pre-incremento: PRIMERO suma 1 a 'a' (pasa a 6), y LUEGO asigna ese 6 a 'b'
console.log(b); // 6
```

```
console.log(a); // 6

// Operadores unarios de signo y conversión:
let s = "25";
console.log(+s); // 25 numérico (equivale a Number(s))
console.log(-s); // -25 numérico
```

2. Operadores de Asignación
Asignan un valor a una variable:

- **Simple (=):** `let x = 10;`
- **Compuestos:** Realizan la cuenta matemática y guardan el resultado de golpe:
  - `x += 5;` equivale a: `x = x + 5;`
  - `x -= 2;` equivale a: `x = x - 2;`
  - `x *= 3;` equivale a: `x = x * 3;`
  - `x /= 2;` equivale a: `x = x / 2;`
  - `x %= 2;` equivale a: `x = x % 2;`
  - `x **= 2;` equivale a: `x = x ** 2;`

Asignaciones Lógicas Modernas (ES2021):

- **??= (Asignación nula lógica):** Solo pone el valor si la variable contiene `null` o `undefined`:

  ```
  let usuario = null;
  usuario ??= "Invitado";
  console.log(usuario); // "Invitado" (se asignó porque era null)

  let puerto = 0;
  puerto ??= 8080;
  console.log(puerto); // 0 (respeta el número cero porque 0 no es null ni undefined)
  ```

- **||= (Asignación lógica OR):** Asigna si la variable contiene cualquier valor falso (`0`, `""`, `null`, `undefined`):

  ```
  let titulo = "";
  titulo ||= "Sin Título";
  console.log(titulo); // "Sin Título" (al ser cadena vacía se sobreescribe)
  ```

3. Operadores de Comparación (Relacionales)

Comprueban dos valores y devuelven siempre un booleano (true o false):

```
// IGUALDAD ABSTRACTA (==) -> Convierte los tipos si son distintos (PELIGROSO)
console.log(5 == "5");  // true (convierte el texto "5" a número y dice que son iguales)

// IGUALDAD ESTRICTA (===) -> Comprueba que coincidan en VALOR y en TIPO
console.log(5 === "5"); // false (uno es un número y el otro es texto)

// DESIGUALDAD ESTRICTA (!==) -> Devuelve true si son distintos en valor o en tipo
console.log(5 !== "5"); // true (son de tipos diferentes)

console.log(5 == "5");   // true (El string se convierte a número)
console.log(5 === "5");  // false (Distinto tipo: number vs string)

console.log(null == undefined);  // true (Tratados como equivalentes por la regla abstracta)
console.log(null === undefined); // false (Distintos tipos de datos)

console.log(0 == false);  // true (0 y false se consideran equivalentes con ==)
console.log(0 === false); // false (number vs boolean)

// MAYOR Y MENOR:
console.log(10 > 5);   // true (10 es mayor que 5)
console.log(5 < 15);   // true (5 es menor o igual a 5)

// MAYOR O IGUAL Y MENOR O IGUAL:
console.log(10 >= 5);   // true (10 es mayor que 5)
console.log(5 <= 5);   // true (5 es menor o igual a 5)
```

4. Operadores Lógicos (Tablas de Verdad)

Los operadores && y | | no devuelven necesariamente valores booleanos, sino el valor exacto del operando que resolvió la expresión:

- **&& (AND / Y):** Da true solo si **ambos lados** son verdaderos. Si el primero es falso, se detiene inmediatamente (cortocircuito):

  ```
  console.log(true && true);   // true
  console.log(true && false);  // false
  console.log(false && "HOLA");// false (al ser el primero false, ni mira el segundo)
  ```
- **|| (OR / O):** Da true si **al menos uno** de los lados es verdadero. Si el primero es verdadero, se detiene de inmediato:

  ```
  console.log(false || true);  // true
  console.log(false || false); // false
  console.log("Texto" || false); // "Texto" (al ser el primero verdadero, lo devuelve)
  ```
- **! (NOT / Negación):** Invierte el valor booleano:

  ```
  console.log(!true);  // false
  ```

```
console.log(!false); // true
console.log(!"hola"); // false (un texto lleno es verdadero; su negación es false)
```

La doble negación (`!!valor`) convierte cualquier dato a su equivalente booleano primitivo.

- **?? (Coalescencia nula):** Devuelve el segundo valor **solo si el primero es null o undefined**:

```
let puntos = 0;
console.log(puntos || 10); // 10 (con || el cero se confunde con falso)
console.log(puntos ?? 10); // 0 (con ?? se respeta el cero como valor válido)

// AND (&&): Devuelve el primer falsy que encuentra o el último valor
console.log("Usuario" && 0 && "Admin"); // 0
console.log("Servidor OK" && 200);      // 200

// OR (||): Devuelve el primer truthy que encuentra o el último valor
console.log(false || "Nombre por defecto"); // "Nombre por defecto"
console.log("Base de datos 1" || "Base 2"); // "Base de datos 1"

// Coalescencia Nula (??): Solo salta si el operando izquierdo es null o undefined
let puertoInput = 0;
let puertoA = puertoInput || 3000; // 3000 (¡Error! Considera el puerto 0 como inválido)
let puertoB = puertoInput ?? 3000; // 0 (Correcto: conserva el 0 porque no es nulo)

// Doble negación (!!): Convierte cualquier expresión a booleano canónico
console.log(!!"texto"); // true
console.log(!!0);       // false
```

Tablas de Verdad: Valores *Truthy* vs. *Falsy*

Para dominar los operadores lógicos (&&, ||, !) y las estructuras de control, se debe memorizar la lista cerrada de los **únicos 8 valores falsos (*falsy*)** que existen en JavaScript. Cualquier otro valor se evalúa siempre como verdadero (*truthy*):

Valores Falsy (se evalúan a false)	**Valores Truthy que suelen confundir al alumno**
false	"0" (cadena con un cero) -> true

0, -0, 0n	"false" (cadena con texto) -> true
"", '', `` (cadena vacía)	[] (array vacío) -> true
null	{} (objeto vacío) -> true
undefined	function(){} -> true
NaN	-1, 3.14 (cualquier número no nulo) -> true

5. Operadores a Nivel de Bits (*Bitwise*)
Operan tratando los números enteros como una hilera de 32 ceros y unos (binario):

// El número 5 en binario es: 0101
// El número 1 en binario es: 0001

Operador	Operación	Ejemplo	Cómputo binario	Resultado
`&`	AND a nivel de bits	`5 & 1`	`0101 & 0001`	`1`
`|`	OR a nivel de bits	`5 | 1`	`0101 | 0001`	`5`
`~`	NOT a nivel de bits	`~5`	Inversión de bits	`-6`
`^`	XOR a nivel de bits	`5 ^ 1`	`0101 ^ 0001`	`4`
`<<`	Desplazamiento a la izquierda	`5 << 1`	Desplaza 1 bit	`10`
`>>`	Desplazamiento a la derecha con signo	`5 >> 1`	Desplaza 1 bit	`2`
`>>>`	Desplazamiento a la derecha sin signo	`5 >>> 1`	Rellena con ceros	`2`

// AND a nivel de bits (&): Solo pone un 1 donde ambos tengan 1
console.log(5 & 1); // Da 1 (0101 & 0001 = 0001)

```
// OR a nivel de bits (|): Pone un 1 si cualquiera de los dos tiene 1
console.log(5 | 1); // Da 5 (0101 | 0001 = 0101)

// Desplazamiento a la izquierda (<<): Mueve los bits hacia la izquierda (multiplica por 2)
console.log(5 << 1); // Da 10
```

6. Operadores Específicos del Lenguaje

A. Operador Condicional Ternario (? :)

Es un if/else resumido en una sola línea:

```
let edad = 19;
// Estructura: (condición) ? valor_si_es_true : valor_si_es_false
let estado = (edad >= 18) ? "Mayor de edad" : "Menor de edad";
console.log(estado); // "Mayor de edad"
```

B. Operador Encadenamiento Opcional (?.)

Evita que el programa se rompa al consultar propiedades anidadas si algo es nulo:

```
let alumno = { nombre: "Carlos", contacto: null };

// Sin ?., esto daría un fallo grave: TypeError: Cannot read properties of null
// Con ?., devuelve undefined de forma segura sin romper el programa:
console.log(alumno.contacto?.telefono); // undefined
```

C. Operador delete

Borra una propiedad de un objeto:

```
let coche = { marca: "Seat", color: "Rojo" };
delete coche.color; // Borra la propiedad color
console.log(coche); // { marca: "Seat" }
```

D. Operador in

Comprueba si un objeto tiene una propiedad concreta:

```
let persona = { nombre: "Marta", edad: 30 };
console.log("nombre" in persona); // true
console.log("apellidos" in persona); // false
```

E. Operador instanceof

Comprueba si un objeto fue creado a partir de una clase o plantilla:

```
let hoy = new Date();
console.log(hoy instanceof Date); // true
```

F. Operador coma (,)

Ejecuta varias instrucciones de izquierda a derecha y se queda con el resultado de la última:

let resultado = (1 + 2, 3 + 4); // Calcula 3, luego calcula 7, y guarda el 7
console.log(resultado); // 7

Es crucial no confundir el *operador* coma con la coma que actúa como *separador de argumentos*.

```
// Caso 1: Separador de argumentos en una función
console.log(1, 2, 3); // Imprime los tres números por separado: 1 2 3

// Caso 2: Operador coma dentro de un solo argumento (gracias a los
paréntesis)

console.log((1, 2, 3)); // Imprime únicamente el último valor: 3
```

A veces se usa para ejecutar una lógica intermedia justo antes de realizar una evaluación en un `if` o un `while`:

```
let x = 0;

// Incrementa x, evalúa x * 2, y el resultado final decide si entra al
IF
if (x++, x * 2 > 0) {
    console.log("Entró al bloque con x valiendo:", x); // x vale 1

}
```

G. Operador void

Evalúa una expresión y descarta su valor devolviendo siempre undefined:

let valor = void(5 + 5);
console.log(valor); // undefined

E. Precedencia y Asociatividad de Operadores

Jerarquía de evaluación de mayor a menor prioridad:

1. **Agrupación:** Paréntesis `( ... )`
2. **Miembros y llamadas:** Acceso a propiedades (`.`, `[ ]`), llamadas a funciones `( )`
3. **Operadores unarios:** `typeof`, `!`, `++`, `--`, `+`, `-`, `delete`, `void`
4. **Exponenciación:** `**` (asociatividad de derecha a izquierda)
5. **Multiplicación, división, resto:** `*`, `/`, `%`
6. **Suma y resta:** `+`, `-`
7. **Desplazamiento de bits:** `<<`, `>>`, `>>>`
8. **Relacionales y pertenencia:** `<`, `<=`, `>`, `>=`, `in`, `instanceof`
9. **Igualdad:** `===`, `!==`, `==`, `!=`

10. **Operadores de bits lógicos:** `&`, `^`, `|`
11. **Lógicos de alta prioridad:** `&&` (conjunción)
12. **Lógicos de baja prioridad:** `||` (disyunción) y `??` (fusión de nulos) — *Nota: Mezclar `||` y `??` sin paréntesis lanza un `SyntaxError`.*
13. **Condicional ternario:** `? :`
14. **Asignación:** `=`, `+=`, `-=`, etc. (asociatividad de derecha a izquierda)
15. **Coma:** `,`

F. Actividades Prácticas y Ejercicios del Criterio 2.2

Ejercicio 1. Evaluación de Cortocircuito y Precedencia: Predice el valor devuelto por las siguientes sentencias antes de ejecutarlas en la consola de las DevTools (F12):

1. `console.log(0 || "Fallback");`
2. `console.log(0 ?? "Fallback");`
3. `console.log(5 + "5");`
4. `let i = 2; console.log(i++ * ++i);`

Ejercicio 2. Actividad Propuesta de los Manuales (Verificación Exhaustiva de Variables y Operadores)

Crea un script llamado verificacion_tipos.js y completa el siguiente bloque de código para analizar en la consola del navegador los comportamientos de asignación, comprobación y conversión de tipos:

```
// 1. Demostración de ámbito de bloque e inmutabilidad
var variableGlobal = "Accesible fuera";
{
  let variableLocal = "Aislada en bloque";
  const CONFIG_APP = { version: "1.0.0", entorno: "test" };

  // Mutabilidad de propiedad en objeto declarado con const
  CONFIG_APP.entorno = "produccion";
  console.log("Objeto modificado:", );
}

// 2. Comprobaciones con el operador typeof
console.log("typeof 50n:", );            // bigint
console.log("typeof Symbol():", );   // symbol
console.log("typeof null:", );          // object (anomalía)
console.log("typeof NaN:", );            // number

// 3. Coerción y comparaciones
console.log("5 == '5':", );              // true (igualdad abstracta)
console.log("5 === '5':",);             // false (igualdad estricta)
console.log("5 + '5':", );               // "55" (concatenación)
console.log("5 - '5':", );               // 0 (conversión numérica)
```

```
// 4. Operadores especiales in, delete e instanceof
const alumno = { id: 1001, curso: "2 DAW", activo: true };
console.log("¿Tiene propiedad 'curso'?:",); // true
delete alumno.activo;
console.log("Objeto tras borrado:", alumno);
console.log("alumno es instancia de Object:", ); // true
```

Ejercicio 3. Validador de Tipos

Se entrega una interfaz interactiva donde el alumno debe introducir expresiones de JavaScript para comprobar en vivo la diferencia entre igualdad abstracta e igualdad estricta, la resolución de evaluaciones en cortocircuito y el resultado devuelto por operadores unarios.

Código Base: laboratorio_operadores.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Laboratorio: Tipos y Operadores en JavaScript</title>
  <style>
    body { font-family: monospace; margin: 30px; background-color: #1e1e1e; color: #d4d4d4;
}
    .panel { max-width: 650px; background: #252526; padding: 25px; border-radius: 8px;
border: 1px solid #3c3c3c; }
    h2 { color: #4ec9b0; margin-top: 0; }
    .fila { margin-bottom: 15px; }
    label { display: block; margin-bottom: 5px; color: #9cdcfe; }
    input[type="text"] { width: 95%; padding: 8px; background: #3c3c3c; border: 1px solid
#555; color: #ce9178; font-size: 14px; }
    button { padding: 8px 16px; background: #0e639c; color: white; border: none; cursor:
pointer; border-radius: 4px; }
    button:hover { background: #1177bb; }
    .salida { margin-top: 20px; padding: 15px; background: #181818; border-left: 4px solid
#4ec9b0; font-size: 13px; line-height: 1.6; }
    .resaltado { color: #dcdcaa; font-weight: bold; }
  </style>
</head>
<body>

  <div class="panel">
    <h2>Laboratorio CE 2.b: Comprobador de Expresiones</h2>

    <div class="fila">
      <label for="campoValor1">Operando A (ej. 0, "0", false, null):</label>
      <input type="text" id="campoValor1" value="0">
    </div>
```

```
    <div class="fila">
      <label for="campoValor2">Operando B (ej. false, "", undefined, "0"):</label>
      <input type="text" id="campoValor2" value="false">
    </div>

    <button id="btnEvaluar">Evaluar Expresiones y Comparaciones</button>

    <div id="pantallaSalida" class="salida">Pulsa el botón para procesar...</div>
  </div>

  <script src="laboratorio_operadores.js"></script>
</body>
</html>
```

Código Base: laboratorio_operadores.js

```
function procesarEvaluacion() {
  const txtA = document.getElementById("campoValor1").value.trim();
  const txtB = document.getElementById("campoValor2").value.trim();
  const visor = document.getElementById("pantallaSalida");

  // Conversión segura de cadenas literales a primitivos reales usando JSON.parse
  let valA, valB;
  try {
    valA = JSON.parse(txtA);
  } catch (e) {
    valA = txtA; // Si no es JSON válido, lo trata como string puro
  }

  try {
    valB = JSON.parse(txtB);
  } catch (e) {
    valB = txtB;
  }

  // Evaluaciones de tipos y operaciones
  const tipoA = typeof valA;
  const tipoB = typeof valB;
  const igualdadAbstracta = (valA == valB);
  const igualdadEstricta = (valA === valB);
  const operacionOr = valA || valB;
  const operacionNullish = valA ?? valB;

  visor.innerHTML = `
    <strong>Operando A:</strong> <span class="resaltado">${String(valA)}</span> (typeof:
${tipoA})<br>
```

```
    <strong>Operando B:</strong> <span class="resaltado">${String(valB)}</span> (typeof: ${tipoB})<br>
    ---------------------------------------------------<br>
    <strong>Igualdad abstracta (A == B):</strong> ${igualdadAbstracta}<br>
    <strong>Igualdad estricta   (A === B):</strong> ${igualdadEstricta}<br>
    <strong>Cortocircuito OR    (A || B):</strong> ${String(operacionOr)}<br>
    <strong>Coalescencia nula   (A ?? B):</strong> ${String(operacionNullish)}
  `;
}

document.getElementById("btnEvaluar").addEventListener("click", procesarEvaluacion);
```

Modificaciones Solicitadas al Alumnado:

1. **Modificación 1 (Incorporación del Operador AND y Negación Booleana):** Modifica el archivo laboratorio_operadores.js para que calcule e imprima también el resultado de evaluar el operador lógico valA && valB y la doble negación booleana !!valA.
2. **Modificación 2 (Protección contra Excedentes de Entero con BigInt):** Añade un bloque condicional que verifique si valA es un valor numérico y supera Number.MAX_SAFE_INTEGER. Si lo supera, debe emitir una advertencia en la pantalla indicando: "Aviso: Precisión comprometida. Utiliza BigInt (sufijo n) para operar en este rango".

Ejercicio 4: Análisis de Precedencia y Operadores Unarios Combinados

Analiza la siguiente secuencia de instrucciones de JavaScript y calcula el valor final que albergan las variables resultado, x e y sin ejecutar el código en el navegador:

```
let x = 3;
let y = 4;
let resultado = x++ * ++y + (x & 1);
```

Ejercicio 5: Comparativa de Cortocircuitos con Valores Falsy Críticos

Una función de configuración recibe un parámetro opcional con la tasa de descuento de un cliente:

```
function fijarDescuento(descuento) {
  let tasaFinalA = descuento || 15;
  let tasaFinalB = descuento ?? 15;
  console.log(`Tasa A: ${tasaFinalA} | Tasa B: ${tasaFinalB}`);
}
```

- Indica razonadamente qué imprimirá la función si un cliente VIP tiene asignado un descuento legítimo del **0%** (fijarDescuento(0)), identificando cuál de las dos asignaciones es la correcta para el sistema.

Ejercicio Tipo Test:

1. ¿Qué valor almacena en memoria una variable declarada con la palabra reservada var si todavía no se le ha asignado ningún dato (var micoche;)?

a) null.

b) undefined.

c) 0.

d) NaN.

2. Si se evalúa la expresión aritmética 12 + "5" en JavaScript, ¿qué resultado exacto se obtiene?

a) 17.

b) NaN.

c) "125".

d) Lanza una excepción de tipo TypeError.

3. ¿Cuál de los siguientes operadores evalúa si dos operandos son estrictamente idénticos, exigiendo que coincidan tanto en valor como en tipo de datos sin permitir conversiones ocultas?

a) ==.

b) =.

c) ===.

d) eq

4. ¿Qué valor devuelve la instrucción typeof null al ejecutarse en el motor del navegador?

a) "null".

b) "undefined".

c) "object".

d) "void"

5. ¿Qué operador introducido en los estándares modernos asigna un valor a una variable única y exclusivamente si dicha variable contiene null o undefined, respetando valores como 0 o false?

a) ||=.

b) &&=.

c) ??=.

d) ?=

6. Si una variable declarada con const contiene una matriz (const notas = [5, 8];), ¿cuál de las siguientes instrucciones es completamente legal y no provocará ningún error de ejecución?

a) notas = [10, 9];

b) notas.push(7);

c) const notas = [4, 6];

d) notas = 10;